ALMA MATER STUDIORUM UNIVERSITÀ DI BOLOGNA
DIPARTIMENTO DI INFORMATICA - SCIENZA ED INGEGNERIA (DISI)
LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

Attività Progettuale in Fondamenti di Intelligenza Artificiale - Classificazione di Immagini tramite Genetic Programming

Studente:
Daniele Guiducci

Docente:
Prof. Andrea Roli
Prof. Federico Chesani

Introduzione e Obiettivi

Obiettivo

- Classificare immagini sintetiche di **cerchi** o **quadrati.**
- Evolvere una funzione matematica capace di effettuare classificazione partendo dalle feature estratte dalle immagini.

$$f(a, b, c, d, ...) \rightarrow \{0, 1\}$$

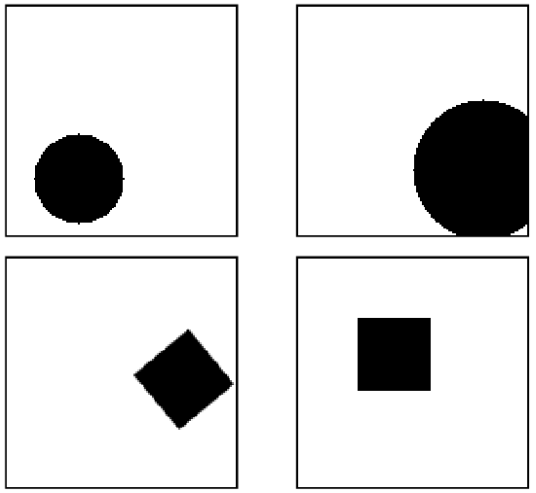
Pipeline

- Creazione del dataset di immagini (cerchi e quadrati, con possibili rotazioni e casi tagliati).
- Estrazione delle feature dalle immagini per generare il file .dat di input per TinyGP (algoritmo evolutivo).
- Riscrittura e adattamento di TinyGP in Java, facendo riferimento al paper di base.
- Test del modello sulle immagini di test e calcolo dell'accuracy.

Dataset ed Estrazione Feature

Descrizione del dataset

- 1000 immagini (700 train, 300 test).
- Cerchi con centro e raggio variabile.
- Quadrati con lato e rotazione casuale (da 0° a 90°).
- Inclusi casi "difficili": figure tagliate.



Estrazione Feature

1. Conversione in scala di grigi
2. Binarizzazione tramite *treshold*.
3. Estrazione del contorno tramite *findContours*.
4. Calcolo delle feature quantitative (Normalizzazione).
5. Memorizzazione delle feature e dell'etichetta della classe.

Algorithm 3 Estrazione delle feature da un'immagine

```
Carica l'immagine
Applica una treshold per ottenere l'immagine binaria
Trova i contorni esterni della figura
if nessun contorno trovato then
    scarta l'immagine
end if
Seleziona il contorno principale
Calcola l'area del contorno
Calcola il perimetro
Determina il rettangolo di bounding (x, y, w, h)
Calcola l'aspect ratio w/h
Calcola l'extent area/(w·h)
Restituisci il vettore delle feature
```

Algoritmo Genetico (1/2)

Rappresentazione individui

- Ogni individuo è un **programma in forma prefissa**, codificato come un **array di caratteri**. L'algoritmo cerca di trovare un'espressione matematica composta da operatori base, che approssimi al meglio il labeling.

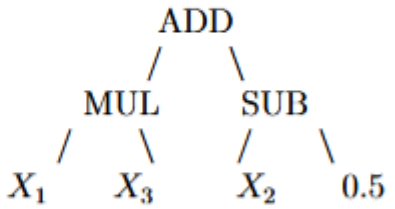
- Il linguaggio è composto da:
  - **Terminali**: feature in ingresso (X1, X2, X3, X4) + eventuali costanti
  - **Funzioni**: operatori aritmetici ADD, SUB, MUL, DIV (divisione protetta).

- Ogni programma rappresenta un **albero sintattico**:
  - **nodi** interni → operatori;
  - **foglie** → variabili o costanti.

Ad esempio, un individuo può essere codificato come:

ADD MUL X1 X3 SUB X2 0.5

che corrisponde all'albero sintattico:



La funzione rappresentata dall'individuo è quindi:

$$f(X_1, X_2, X_3) = X_1 \cdot X_3 + (X_2 - 0.5)$$

Calcolo della fitness

- Ogni individuo prende in input le feature normalizzate di un'immagine e produce un valore continuo. Il valore viene trasformato tramite la **sigmoide** ottenendo una probabilità di appartenenza alle classi.

- La fitness è data dalla somma degli errori assoluti rispetto ai target: più l'individuo predice correttamente, minore è la fitness (Viene usata –fitness per ottenere un problema di massimizzazione).

$$\hat{y} = \frac{1}{1 + e^{-f(X_1, X_2, \ldots, X_n)}}$$

$$\text{fitness} = -\sum_{i=1}^{N} |\hat{y}_i - y_i|$$

Algoritmo Genetico (2/2)

Generazione popolazione iniziale e Ciclo evolutivo

- Popolazione iniziale generata con operatore *grow* fino a una profondità massima (4 livelli). Per ogni individuo (5000):
  - Generazione casuale dell'albero sintattico.
  - Calcolo immediato della fitness.

- Operatori evolutivi
  - **Selezione** (Tournament Selection) tra TSIZE individui: scelto il migliore come genitore e selezione negativa per quello da rimpiazzare
  - **Crossover**: Scelta di un punto casuale nei due genitori, scambio dei sottoalberi selezionati e creazione di un nuovo programma combinato.
  - **Mutazione**: Ogni nodo muta con probabilità PMUT_PER_NODE. Se è un terminale → rimpiazzato con altro terminale. Se è un Operatore → sostituito da altro operatore aritmetico (ADD, SUB, MUL, DIV).

- TinyGP aggiorna la popolazione uno individuo alla volta (**steady-state**), non sostituisce l'intera popolazione a ogni generazione.
- Il processo continua per un numero prefissato di generazioni (800 generazioni) oppure fino a quando l'errore diventa trascurabile.
- Quando un individuo raggiunge una fitness sufficientemente elevata (errore quasi nullo), l'algoritmo memorizza la funzione evoluta nel file *best_program.txt*, usato dal classificatore per leggere la formula.

Classificazione e Risultati

Classificazione

- Legge tutte le immagini presenti nella cartella indicata.

- Per ciascuna immagine:
  - **estrae** le feature con la stessa procedura descritta nell'algoritmo
  - **applica** la funzione di classificazione
  - **assegna** la label in base alla probabilità ottenuta.

- Salva tutti i risultati in un **DataFrame** e in un file CSV (classification_results.csv) per consentire il calcolo dell'accuracy.

Algorithm 4 Classificazione di un singolo esempio tramite GP

Require: Feature X_1, X_2, X_3, X_4; formula GP $gp_formula$
Ensure: Classe predetta

```
Valuta la formula GP usando i valori X1, X2, X3, X4
if si verifica un errore o il risultato non è numerico then
    result 0.0
end if
Applica la funzione sigmoide al risultato: output = 1/(1+e^(-result))
if output ≥ 0.5 then Label predetta = "quadrato" else Label predetta = "cerchio"
return output
```

Risultati

$$\text{Accuracy} = \frac{\text{n° di immagini correttamente classificate}}{\text{n° totale di immagini}} \cdot 100$$

```
==================================================================
Formula GP caricata:
((((((X2 / (((X2 / (X3 + (X1 + ((X2 * X4) - ((X2 + X4) - X4))))) - X
 - X3))))) - X3) - ((1.85 / X3) / -0.06)) + (X2 + (X4 - X4))) / ((((
) - 0.85) / (X3 + ((X2 / ((X3 / X2) / X2)) / X4)))) / ((X2 * ((1.85
))
==================================================================

Risultati salvati in classification_results.csv

==================================================
Accuratezza complessiva: 96.33% (289/300)
==================================================
```

Conclusioni

Conclusioni

- È stato sviluppato un sistema di classificazione cerchi/quadrati basato su Genetic Programming, implementato tramite **TinyGP**.
- TinyGP ha gestito l'intero processo evolutivo: generazione popolazione, fitness, selezione, crossover e mutazione.
- Il modello ha evoluto programmi simbolici capaci di combinare le feature e minimizzare l'errore di classificazione.
- Il miglior programma è stato testato dopo normalizzazione e applicazione della sigmoide, raggiungendo accuratezza >90% su vari dataset.

Limiti

- Feature semplici e sensibili a rumore o deformazioni.
- TinyGP limita la complessità dei modelli evolvibili e dipende fortemente dai parametri iniziali.

Sviluppi futuri

- Uso di feature più ricche, dataset più complessi e classificazione multi-classe..